

Ingeniería en Informática

UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS (CUCEI)

Arquitectura de computadoras

Maestro. Jorge Ernesto Lopez Arce Delgado

Propuesta de Algoritmo

10/05/26

INTRODUCCIÓN

Como parte del proyecto final, se propone desarrollar un programa en Python capaz de traducir instrucciones en ensamblador MIPS tipo R hacia código máquina binario.

El objetivo de este programa es generar instrucciones binarias que posteriormente serán cargadas en la memoria de instrucciones del datapath implementado en Verilog, permitiendo verificar el correcto funcionamiento de cada parte del programa.

DESCRIPCIÓN

El programa recibirá como entrada una instrucción escrita en ensamblador MIPS, por ejemplo:

add \$t2, \$t0, \$t1

Posteriormente el programa lo que hará es:

- Separar la operación y los registros.
- Identificará el tipo de instrucción.
- Buscará el código correspondiente de:
 - opcode
 - registros
 - funct
- Construirá la instrucción de 32 bits.
- Mostrará el resultado en binario.

FORMATO DE INSTRUCCIONES TIPO R

Las instrucciones tipo R en MIPS utilizan el siguiente formato

opcode	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

* Todas las instrucciones utilizaran **opcode=000000**

Campo	Valor
opcode	000000
rs (\$t0)	01000
rt (\$t1)	01001
rd (\$t2)	01010
shamt	00000
funct (add)	100000

Y el código en máquina que se generaría es:

00000001000010010101000000100000

Instrucciones tipo R

Instrucción	Funct (hex)	Operación	Sintaxis
<code>add</code>	0x20	$\$rd = \$rs + \$rt$	<code>add \$rd, \$rs, \$rt</code>
<code>sub</code>	0x22	$\$rd = \$rs - \$rt$	<code>sub \$rd, \$rs, \$rt</code>
<code>and</code>	0x24	$\$rd = \$rs \& \$rt$	<code>and \$rd, \$rs, \$rt</code>
<code>or</code>	0x25	$\$rd = \$rs \$rt$	<code>or \$rd, \$rs, \$rt</code>
<code>slt</code>	0x2A	$rd = (rs < rt)$	<code>slt \$rd, \$rs, \$rt</code>
<code>nop</code>	0x00	No opera (sll \$0,\$0,0)	<code>nop</code>
<code>nor</code>	0x27	$rd = (rs rt)$	<code>nor \$rd, \$rs, \$rt</code>
<code>xor</code>	0x26	$\$rd = \$rs \wedge \$rt$	<code>xor \$rd, \$rs, \$rt</code>
<code>sll</code>	0x00	$\$rd = \$rt \ll shamt$	<code>sll \$rd, \$rt, shamt</code>
<code>srl</code>	0x02	$\$rd = \$rt \gg shamt$	<code>srl \$rd, \$rt, shamt</code>
<code>sltu</code>	0x2B	$rd = (rs < rt)$ sin signo	<code>sltu \$rd, \$rs, \$rt</code>

ALGORITMO PROPUESTO

1. Leer instrucción ensamblador
2. Separar operación y registros
3. Identificar la instrucción
4. Buscar opcode correspondiente
5. Buscar registros en tabla de registros
6. Buscar código func. correspondiente
7. Construir instrucción binaria de 32 bits
8. Mostrar resultado
9. Finalizar programa

El código máquina generado por el programa será utilizado para poder precargar memoria de instrucciones, verificar señales de control, validar operaciones de la ALU y comprobar la ejecución correcta del datapath

Esto permitirá realizar pruebas funcionales del procesador MIPS implementado en verilog.

CONCLUSIÓN

La propuesta se centra más que nada en desarrollar un ensamblador básico el cual utilice variables, cadenas, separación de texto y concatenación de bits, permitiendo automatizar la generación de código máquina binario y facilitar las pruebas del procesador desarrollado durante este proyecto.